Unifying Environmental, Social, and Governance (ESG) Metrics with Financial Analysis

DANL 299 Project

AUTHOR
Byeong-Hak Choe

AFFILIATION
SUNY Geneseo

PUBLISHED
February 27, 2026

Project Submissions

Please submit the following three items for your project:

1. A Python Script File (*.py) for Data Collection

Submit the scripts used to collect your data.

- `danl_299_LASTNAME_FIRSTNAME_stock_data_collection.py`

2. A CSV File (*.csv)

Export your final DataFrames as CSV files using the Python scripts above

- `danl_299_LASTNAME_FIRSTNAME_stock.csv`

Contains daily stock market data from the beginning of 2024 to March 31, 2025, with the following columns:

- `Symbol` : Company's stock ticker
- `Name` : Company name
- `Date` : Trading date
- `Open` : Opening price on the date
- `High` : Highest price on the date
- `Low` : Lowest price on the date
- `Close` : Closing price on the date

↑ Back to top

- `Adj Close` : Adjusted close price accounting for splits and dividends
- `Volume` : Trading volume
- `Dividend` : Cash dividend paid per share on the date (if any), as reported by Yahoo Finance

Note: A CSV file must not exceed 30 MB.

3. Jupyter Notebook File (*.ipynb) for Your Project Webpage

Filename: `danl_299_LASTNAME_FIRSTNAME_stock_ESG.ipynb`

Include your full Python code;(excluding the data collection scripts) and your project explanation, written clearly in text cells.

File size limit: 30 MB

Note: For data loading in Google Colab, you may use the `file.upload()` function.

- This lecture slide could be useful for data loading in Google Colab.

Project Webpage Publication

In addition to submitting your files:

Publish your final project as a webpage on your personal website

Add a working link to the project in your site's navigation bar

This must also be completed by the deadline below

Deadline: Sunday, March 1, 11:59 P.M.

Project

1. Collect the following data from Yahoo! Finance:
   - Environmental, Social, and Governance (ESG) data
   - Stock market data
2. Publish a webpage presenting your data analysis project on your personal website, hosted via GitHub.
   - Your analysis should center around the agenda: **"Unifying ESG Metrics with Financial Analysis"**

- Present your work using a **Jupyter Notebook**.
- **Due Date:** <u>December 16, 2025 (Friday) by 11:59 P.M.</u>

↑ Back to top

Project Data

- Below is the `esg_proj_2024_data` and `esg_proj_2025` DataFrames.

1. `esg_proj_2024_data`

- The `esg_proj_2024_data` DataFrame which provides a list of companies and associated information, including ESG scores.

```
import pandas as pd
url_2024 = "https://bcdanl.github.io/data/esg_proj_2024_data.csv"
esg_proj_2024_data = pd.read_csv(url_2024)
```

Year <dbl>	Symbol <chr>	Name <chr>
2024	A	Agilent Technologies Inc. Common Stock
2024	AA	Alcoa Corporation Common Stock
2024	AAL	American Airlines Group Inc. Common Stock
2024	AAP	Advance Auto Parts Inc.
2024	AAPL	Apple Inc. Common Stock
2024	ABBV	AbbVie Inc. Common Stock
2024	ABT	Abbott Laboratories Common Stock
2024	ACGL	Arch Capital Group Ltd. Common Stock
2024	ACM	AECOM Common Stock
2024	ACN	Accenture plc Class A Ordinary Shares (Ireland)

1-10 of 625 rows | 1-3 of 13 columns Previous 1 2 3 4 5 6 ... 63 Next

Variable Description

- `Symbol` : a company's ticker;
- `Company Name` : a company name;
- `Sector` : a sector a company belongs to;
- `Industry` : an industry a company belongs to;
- `Country` : a country a company belongs to;
- `Market_Cap` : a company's market capitalization as of December 20, 2024 (Source: Nasdaq's Stock Screener).
  - A company's market capitalization is the value of the company that is traded on the stock market, calculated by multiplying the total number of shares by the present share price.
- `IPO_Year` : the year a company first went public by offering its shares to be traded on a stock exchange.
- `Total_ESG` : The overall ESG (Environmental, Social, and Governance) risk score, summarizing the company's exposure to ESG-related risks a [illegible] 024. A lower score indicates lower risk.
- `Environmental` : The company's exposure to environmental risks (e.g., emissions, energy use, environmental policy) as of March 31, 2024. A lower score indicates lower risk.

↑ Back to top

- `Social` : The company's exposure to social risks (e.g., labor practices, human rights, diversity, and customer relations) as of March 31, 2024. A lower score indicates lower risk.
- `Governance` : The company's exposure to governance-related risks (e.g., board structure, executive pay, shareholder rights, transparency) as of March 31, 2024. A lower score indicates lower risk.
- `Controversy` : A score reflecting the severity of recent ESG-related controversies involving the company as of March 31, 2024. Higher scores typically indicate greater or more serious controversies.

2. `esg_proj_2025`

- The `esg_proj_2025` DataFrame provides a list of companies and associated information.

```
url_2025 = "https://bcdanl.github.io/data/esg_proj_2025.csv"
esg_proj_2025 = pd.read_csv(url_2025)
```

Year <dbl>	Symbol <chr>	Name <chr>
2025	A	Agilent Technologies Inc. Common Stock
2025	A	Agilent Technologies Inc. Common Stock
2025	AA	Alcoa Corporation Common Stock
2025	AA	Alcoa Corporation Common Stock
2025	AAL	American Airlines Group Inc. Common Stock
2025	AAMI	Acadian Asset Management Inc. Common Stock
2025	AAON	AAON Inc. Common Stock
2025	AAP	Advance Auto Parts Inc.
2025	AAPL	Apple Inc. Common Stock
2025	AAT	American Assets Trust Inc. Common Stock

1-10 of 3,139 rows | 1-3 of 8 columns Previous 1 2 3 4 5 6 ... 314 Next

Variable Description

- `Market_Cap` : a company's market capitalization as of March 29, 2025.

3. `stock_history_2023`

↑ Back to top

- The `stock_history_2023` DataFrame contains daily historical stock market data for the year 2023.

```
url = "https://bcdanl.github.io/data/stock_history_2023.csv"
stock_history_2023 = pd.read_csv(url)
```

Date	Year	Symbol	Open	High	Low	Close
<date>	<dbl>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
2023-12-29	2023	A	138.587233	139.215036	137.879691	138.547363
2023-12-28	2023	A	139.815657	140.084270	138.930245	139.049637
2023-12-27	2023	A	139.059578	139.437624	138.363189	139.099380
2023-12-26	2023	A	138.591994	139.746018	138.373126	139.089416
2023-12-22	2023	A	138.890449	139.636584	138.074668	138.850662
2023-12-21	2023	A	138.661642	138.950141	137.288750	138.223907
2023-12-20	2023	A	139.069522	140.512061	137.428032	137.467819
2023-12-19	2023	A	137.796117	139.169010	136.612263	139.069519
2023-12-18	2023	A	136.880855	137.477769	136.154622	136.821167
2023-12-15	2023	A	136.532679	138.184112	135.577620	136.075043

1-10 of 10,000 rows | 1-8 of 10 columns Previous 1 2 3 4 5 6 ... 1000 Next

Variable Description

- `Symbol` : Company's stock ticker
- `Date` : Trading date
- `Year` : Trading year
- `Open` : Opening price on the date
- `High` : Highest price on the date
- `Low` : Lowest price on the date
- `Close` : Closing price on the date
- `Volume` : Trading volume
- `Dividend` : Cash dividend paid per share on the date (if any), as reported by Yahoo Finance
- `Stock_Split` : The ratio of any stock split that occurred on the given date.

Project Tasks

1. Data Collection

For data collection, include only the companies that are common to both the `esg_proj_2024_data` and `esg_proj_2025` DataFrames.

↑ Back to top

- Scraping web data falls into a legal gray area. In the U.S., scraping publicly available information is not illegal, but it is not always clearly allowed either.

   - Most companies do not go after individuals for minor or non-commercial violations of their Terms of Service (ToS). Still, if the scraping causes harm, it can lead to legal trouble.
- Tips for Collecting Data from Yahoo! Finance:
   1. Scrape at a reasonable and moderate rate. To avoid overloading servers, use `time.sleep(random.uniform(5, y))` between page visits.
   2. The method of explicit waits are not required, but they are helpful for ensuring elements load before scraping.
   3. Be aware that some companies may not have data available for Environmental, Social, or Governance Risk Scores, or Controversy Level.
   4. Consider starting with the following setup for Selenium web-scrapping

```
# %%
# =============================================================================
# Setup libraries
# =============================================================================
import time
import random
import pandas as pd
import os
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# %%
# =============================================================================
# Setup working directory
# =============================================================================
wd_path = 'PATHNAME_OF_YOUR_WORKING_DIRECTORY'
os.chdir(wd_path)


# %%
# =============================================================================
# Setup WebDriver with options
# =============================================================================
options = Options()
options.add_argument("window-size=1400,1200")  # Set window size
options.add_argument('--disable-blink-features=AutomationControlled')  # Prevent detectio
options.page_load_strategy = 'eager'  # Load only essential content first, skipping non-c

driver = webdriver.Chrome(options=options)
```

↑ Back to top

a. ESG Data

- For each company in the `esg_proj_2025` DataFrame, employ the Python `selenium` library to gather **ESG Risk Ratings**, along with the **Controversy Level** from the **Sustainability** section of each company's webpage on Yahoo! Finance, such as:
  - Apple Inc. (AAPL)
  - Microsoft Corporation (MSFT)

b. Historical Stock Data

- For each company found in both the `esg_proj_2024_data` and `esg_proj_2025` DataFrames, employ the `selenium` library to retrieve:
  - Daily stock prices from January 1, 2024, to March 31, 2025
    - e.g., https://finance.yahoo.com/quote/MSFT/history/?p=MSFT&period1=1704067200&period2=1743446400
    - 1704067200 = January 1, 2024
    - 1743446400 = March 31, 2025
- **Note:** GOOGLEFINANCE function in Google Sheets is freely available for retrieving current or historical stock data.
  - Although our course does not cover Google Sheets, you are welcome to use it to collect historical stock data if you prefer.
  - If you choose Google Sheets' `GOOGLEFINANCE()` for collecting historical stock data, please share your Google Sheets with Prof. Choe.

Dividend Cleaning

If you scrape historical data tables from each company's page on Yahoo Finance (e.g., MSFT Historical Data), you can construct a DataFrame similar to the `df_all` DataFrame shown below.

The `df_all` DataFrame contains stock data for Apple Inc., Microsoft, and Nvidia from the beginning of 2024 through the end of Q1 2025:

```
import pandas as pd
df_all = pd.read_csv('https://bcdanl.github.io/data/aapl_msft_nvda_2024_2025.csv')
```

Symbol <chr>	Date <chr>	Open <chr>	High <chr>	Low <chr>	Close <chr>
AAPL	28-Mar-25	221.67	223.81	217.68	217.9
AAPL	27-Mar-25	221.39	224.99	220.56	223.85
AAPL	26-Mar-25	223.51	225.02	220.47	221.53
AAPL	25-Mar-25	220.77	224.1	220.08	223.75
AAPL	24-Mar-25	221	[illegible]	218.58	220.73
AAPL	21-Mar-25	211.56	218.84	211.28	218.27
AAPL	20-Mar-25	213.99	217.49	212.22	214.1

↑ Back to top

Symbol	Date	Open	High	Low	Close
<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
AAPL	19-Mar-25	214.22	218.76	213.75	215.24
AAPL	18-Mar-25	214.16	215.15	211.49	212.69
AAPL	17-Mar-25	213.31	215.22	209.97	214

1-10 of 949 rows | 1-6 of 8 columns Previous 1 2 3 4 5 6 ... 95 Next

Note that some rows in the `df_all` DataFrame include dividend declarations rather than price and volume data (e.g., Apple Inc. on February 10, 2025; Microsoft on February 20, 2025). These dividend entries appear in the Open/High/Low/Close/Adj Close/Volume columns as strings like "0.25 Dividend".

- Note: Apple Inc's "0.25 Dividend" means that on that specific date, Apple Inc issued a cash dividend of $0.01 per share.

To separate these dividend entries from the actual stock trading data, we use the `str.contains()` method:

```
# Filter rows where the 'Open' column contains the word 'Dividend' (these represent divid
df_dividend = df_all[df_all['Open'].str.contains('Dividend', na=False)]

# Filter out dividend rows to keep only stock price data
df_stock = df_all[~df_all['Open'].str.contains('Dividend', na=False)]
```

- At this point:
  - `df_stock` does not include dividend announcements.
  - `df_dividend` includes only dividend announcements.

We now clean and format the dividend information:

```
# Select only relevant columns for dividend data
df_dividend = df_dividend[['Date', 'Symbol', 'Open']]

# Copy 'Open' column (which contains dividend information) into a new column named 'Divid
df_dividend['Dividend'] = df_dividend['Open']

# Keep only the necessary columns: Date, Symbol, and Dividend
df_dividend = df_dividend[['Date', 'Symbol', 'Dividend']]

# Remove the text " Dividend" from the Dividend column to isolate the numeric value
df_dividend['Dividend'] = df_dividend['Dividend'].str.replace(' Dividend', '')
```

↑ Back to top

Stock Split Cleaning

Similarly, some row in the `df_stock` DataFrame from the "Dividend Cleaning" subsection includes stock splits rather than price and volume data (e.g., Nvidia on June 10, 2024). These stock split entries appear in the Open/High/Low/Close/Adj Close/Volume columns as strings like "10:1 Stock Splits".

To separate these stock split entries from the actual stock trading data, again we use the `str.contains()` method:

```
# Filter rows where the 'Open' column contains the word 'Split' (these represent stock sp
df_split = df_stock[df_stock['Open'].str.contains('Split', na=False)]

# Filter out dividend rows to keep only stock price data
df_stock = df_stock[~df_stock['Open'].str.contains('Split', na=False)]
```

- At this point:
    - `df_stock` includes only daily stock trading information.
    - `df_split` includes only stock splits.

We now clean and format the split information:

```
# Select only relevant columns for dividend data
df_split = df_split[['Date', 'Symbol', 'Open']]

# Copy 'Open' column (which contains dividend information) into a new column named 'Split
df_split['Split'] = df_split['Open']

# Keep only the necessary columns: Date, Symbol, and Split
df_split = df_split[['Date', 'Symbol', 'Split']]

# Remove the text " Stock Splits" from the Split column to isolate the numeric value
df_split['Split'] = df_split['Split'].str.replace(' Stock Splits', '')
```

- Note: NVDA's "10:1 Stock Split" means that each existing share was split into 10 shares.
    - If you owned 1 share before June 10, 2024, you would own 10 shares after the split.
    - To maintain consistency, Yahoo Finance retroactively adjusts all historical prices and volumes to reflect stock splits.
- For example, the table below shows NVDA's adjusted stock prices and volumes from June 7–11, 2024:

Symbol <chr>	Date <chr>	Open <chr>	High <chr>	Low <chr>
NVDA	11-Jun-24	121.77	122.87	118.74
NVDA	10-Jun-24	10:1 Stock Splits	10:1 Stock Splits	10:1 Stock Splits
NVDA	10-Jun-24	120.37	[illegible]	117.01
NVDA	7-Jun-24	119.77	[illegible]	118.02

↑ Back to top

4 rows | 1-5 of 8 columns

Extracting the year from a datetime variable

- To extract the year from a datetime variable in a pandas DataFrame, you can use the `.dt.year` accessor.

```
# Sample DataFrame with string dates
data = {
    'Symbol': ['AAPL', 'MSFT', 'GOOG'],
    'Date': ['2024-12-29', '2024-12-30', '2025-01-03'],
    'Close': [130.21, 265.78, 122.34]
}

df = pd.DataFrame(data)

# Convert 'Date' column to datetime
df['Date'] = df['Date'].astype('datetime64[ns]')

# Extract year from 'Date' variable
df['Year'] = df['Date'].dt.year
```

2. Data Analysis

- If you are unable to complete the data collection tasks, please use the `esg_proj_2024_data` and `stock_history_2023` DataFrames for your data analysis.
- If you successfully completed the data collection tasks, you may "optionally" incorporate the `stock_history_2023` DataFrame into your analysis as an additional data source.
- Below are the key components in the data analysis webpage.
  1. **Title**: A clear and concise title that gives an idea of the project topics.
  2. **Introduction**:
     - **Background**: Provide context for the research questions, explaining why they are significant, relevant, or interesting.
     - **Statement of the Problem**: Clearly articulate the specific problem or issue the project will address.
  3. **Data Collection**: Use a Python script (↑ Back to top e code and the comment on how to retrieve ESG data and historical stock data using Python `selenium`.

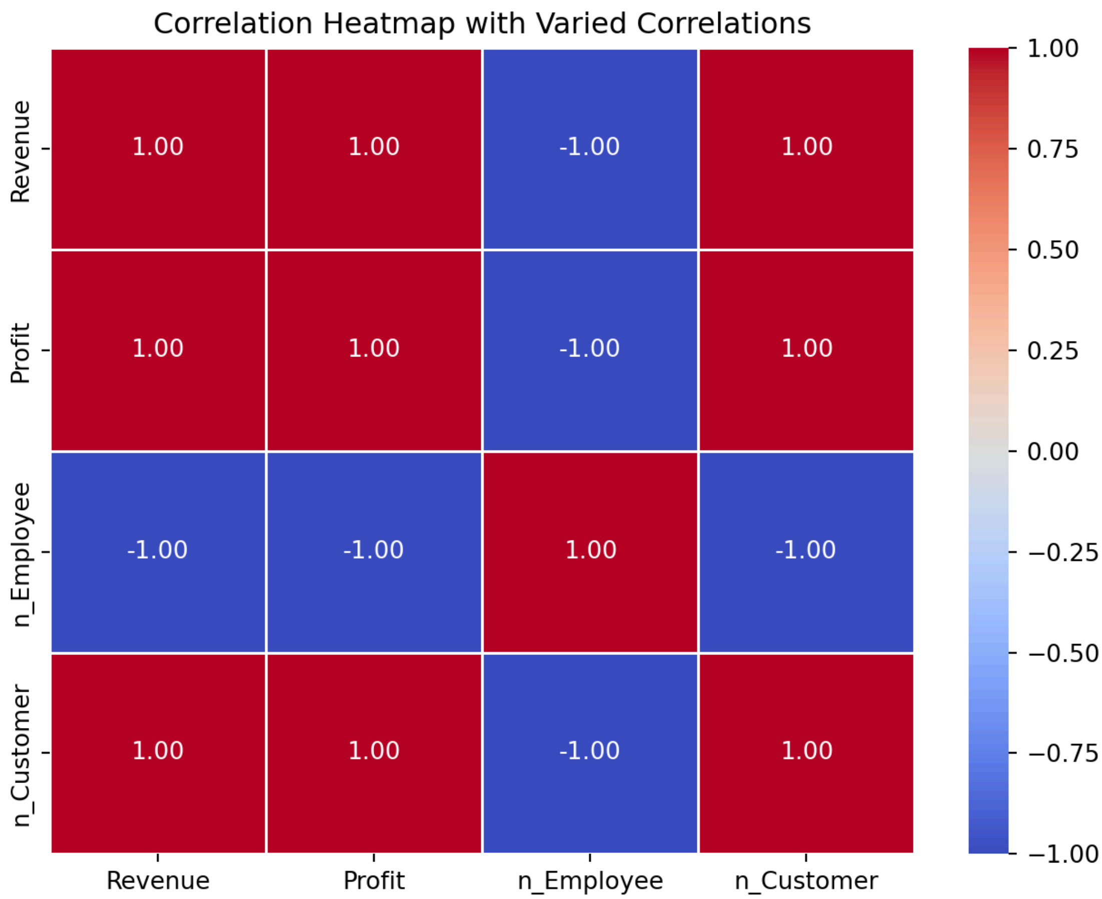
- Do NOT provide your code for data collection in your webpage. You should submit your Python script for data collection to Brightspace.

4. **Descriptive Statistics**

- Provide both grouped and un-grouped descriptive statistics and distribution plots for the ESG data and the finance/accounting data
- Optionally, provide correlation heat maps using `corr()` and `seaborn.heatmap()`. Below provides the Python code for creating a correlation heatmap.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Sample DataFrame with varied correlations
data = {
    'Revenue': [100, 200, 300, 400, 500],
    'Profit': [20, 40, 60, 80, 100],
    'n_Employee': [50, 45, 40, 35, 30],
    'n_Customer': [10, 11, 12, 13, 14]
}

# Create a DataFrame from the dictionary
df = pd.DataFrame(data)

# Calculate the correlation matrix of the DataFrame
corr = df.corr()

# Set up the matplotlib figure size
plt.figure(figsize=(8, 6))

# Generate a heatmap in seaborn:
# - 'corr' is the correlation matrix
# - 'annot=True' enables annotations inside the squares with the correlation values
# - 'cmap="coolwarm"' assigns a color map from cool to warm (blue to red)
# - 'fmt=".2f"' formats the annotations to two decimal places
# - 'linewidths=.5' adds lines between each cell
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)

# Title of the heatmap
plt.title('Correlation Heatmap with Varied Correlations')

# Display the heatmap
plt.show()
```

↑ Back to top



5. **Exploratory Data Analysis**:
    - Explore the trend of ESG scores from 2024 to 2025.
    - Additionally, list the questions you aim to answer.
    - Address the questions through data visualization with `seaborn` (or `lets-plot`) and `pandas` methods and attributes.
6. **Significance of the Project**:
    - Explain its implications for real-world applications, business strategies, or public policy.
7. **References**
    - List all sources cited in the project.
    - Leave a web address of the reference if that is from the web.
    - Indicate if the code and the write-up are guided by generative AI, such as ChatGPT. There will be no penalties on using any generative AI.
    - Clearly state if the code and the write-up result from collaboration with colleagues. There will be no penalties for collaboration, provided that the shared portions are clearly indicated.

↑ Back to top

Interpreting ESG Data on Yah[...]

In **Yahoo Finance**, the **ESG data** helps investors evaluate a company's sustainability profile and exposure to long-term environmental, social, and governance risks. Here's how to interpret each metric:

🔢 Total ESG Risk Score

- **What it means**: A composite score reflecting the company's overall **exposure to ESG-related risks**.
- **How to interpret**:
  - **Lower score = lower risk** → Better ESG performance.
  - **Higher score = higher risk** → More vulnerable to ESG-related issues.
  - *Example*: A company with **total_ESG = 15** is considered to have **lower ESG risk** than one with **total_ESG = 30**.

🌎 Environmental Risk Score

- **What it measures**: Exposure to **environmental risks** such as:
  - Carbon emissions
  - Energy efficiency
  - Waste management
  - Climate change strategy
- **Interpretation**:
  - Lower score → better environmental practices.
  - Higher score → more environmental liabilities or poor sustainability measures.

👥 Social Risk Score

- **What it measures**: Exposure to **social risks**, including:
  - Labor practices
  - Human rights
  - Inclusive culture and representation
  - Customer and community relations
- **Interpretation**:
  - Lower score = better social responsibility.
  - Higher score = more risk from labor issues, PR problems, etc.

🏛️ Governance Risk Score

- **What it measures**: Exposure to **governance-related risks**, such as:
  - Board structure and independence
  - Executive compensation
  - Shareholder rights
  - Transparency and ethics
- **Interpretation**:
  - Lower score suggests better corporate oversight.
  - Higher score suggests poor governanc

↑ Back to top

🚨 Controversy Level

- **What it measures**: Reflects **recent ESG-related controversies** involving the company.
- **Scale**: Usually ranges from **0 (no controversies)** to **5 (severe and ongoing issues)**.
- **Interpretation**:
  - **Low score (0–1)**: Minimal or no controversies.
  - **High score (4–5)**: Major controversies — potential reputational or legal risks.
  - *Note*: A company may have good ESG scores but still be flagged due to a high controversy score.

🧠 ESG Score Summary

Metric	Good Score	Bad Score
total_ESG	Low	High
Environmental	Low	High
Social	Low	High
Governance	Low	High
Controversy	0–1	4–5

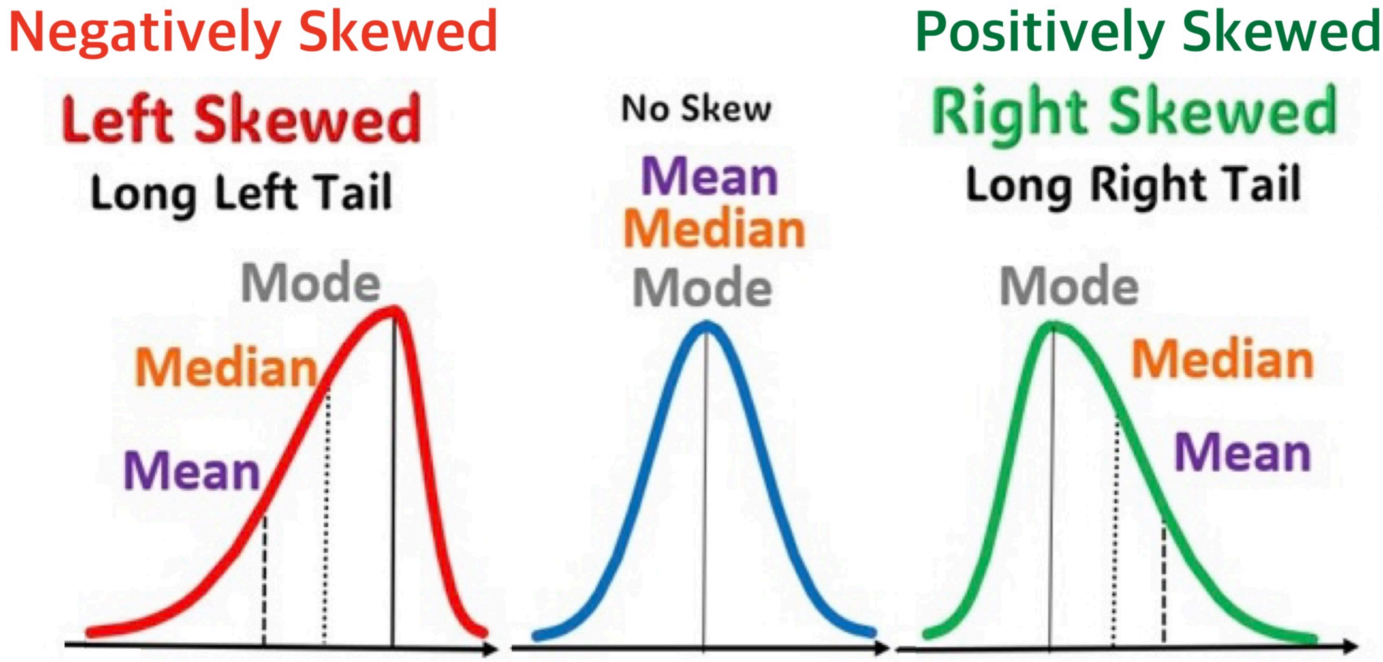
General Tips on Data Visualization 📈

Distribution

When describing the distribution of a variable, we are typically interested in several key characteristics:

1. **Center**: The central tendency of the data, such as the **mean** or **median**, which indicates the **typical or average value**.
2. **Spread**: How spread the values are within the variable, showing the **range** and **standard deviation** of values.
3. **Common Values**: Identifying frequent values and the **mode**.
4. **Rare Values**: Recognizing unusual or infrequent values.
5. **Shape**: The overall shape of the distribution, such as whether it's symmetric, skewed left or right, or having multiple groups with multiple peaks.

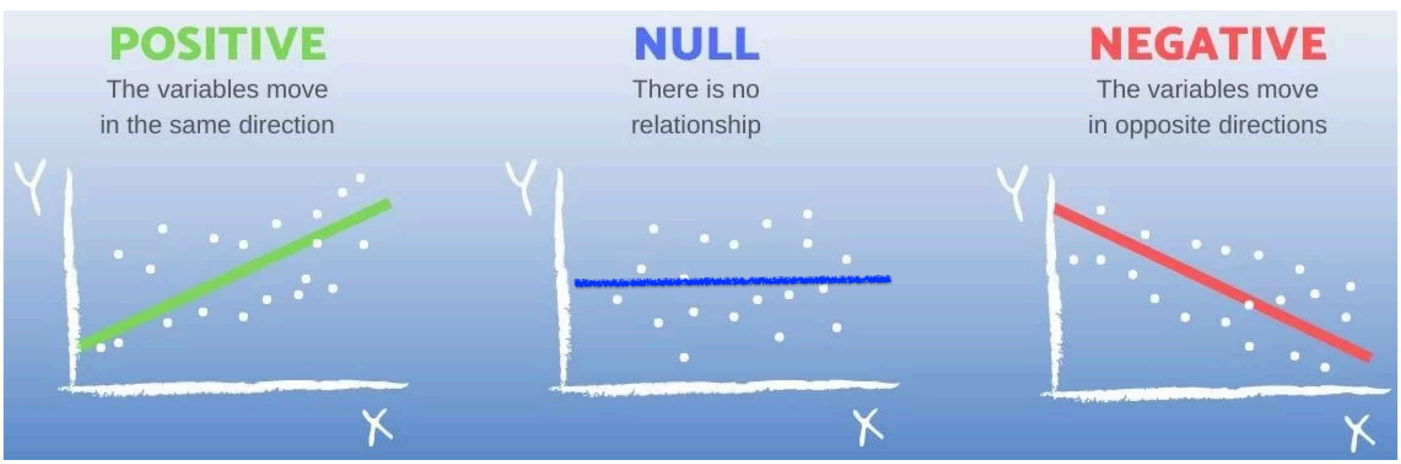
↑ Back to top



Relationship Between Two Variables

1. Start with determining whether the two variables have a **positive association**, a **negative association**, or **no association**.

- E.g., A negative slope in the fitted line indicates that sales decrease as the price increases, while a positive slope would indicate that sales increase with price. A zero slope means that there is no relationship between sales and price; changes in price do not affect sales.



2. **Input on the x-axis; output on the y-axis**

- By convention, the input (or predictor) variable is plotted on the x-axis, and the output (or response) variable on the y-axis.
- This helps visualize potential relationships … s correlation, not necessarily causation.
- Correlation does not necessarily mean cau…

↑ Back to top

3. When a question asks you to describe how the relationship varies by another categorical variable, examine both **the direction of the slope** (negative, positive, or none) from the fitted line and **the steepness of the slope** (steep or shallow).

- The slope of the fitted straight line is the rate at which the "y" variable (like grades) changes as the "x" variable (like study hours) changes. In simple terms, it shows how much one thing goes up or down when the other thing changes.
- For example, a comment such as, "The plot shows a negative relationship between sales and price" does not address how the relationship differs by brand.

4. **The focus is on the relationship, not the distribution.**

- While adding a comment on the distribution of a single variable can be helpful, the question is primarily about the relationship between the two variables.

Time Trend of a Variable

Here are some general tips for describing the time trend of a variable:

1. **Start with Identifying the Overall Trend**

- Look for **the general direction of the trend over time**.
  - Is it moving **upward**, **downward**, or **remaining relatively constant**?

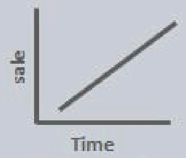
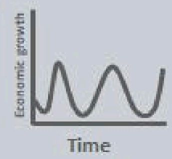
2. **Note Patterns and Cycles**

- Identify any repeating patterns, such as **seasonal fluctuations** (e.g., monthly or quarterly changes) or **long-term cycles**.
  - These can reveal consistent influences that affect the variable over time.

3. **Highlight Any Significant Fluctuations**

- Describe any **sharp increases, decreases**, or **irregular spikes** in the data.

↑ Back to top

Topic	Trend	Seasonality	Cyclic	Irregularity
Fixed Time Interval	Yes	Yes	No	No
Movement Type	Long/Short Term	Short term	Long/Short Term	Random/Irregular
Graph	 Positive Trend	 Seasonality	 Cyclic	 Irregular
Example	Growth in sales over time	Growth of ice-cream sales every summer	Fluctuation of economic growth	Price of the stock over time

Interpreting Visualization

- Be specific.
  - Avoid vague statements. Below examples do not actually explain what the patterns are.
    - "The plot shows how the time trend of a stock price varies across sectors, with each sector having a unique best fitting line and scatter pattern"
    - "The trend shows the evolution of stock price in the market over time"
  - Clearly describe what is the pattern—and how it differs across categories.
- Add Narration:
  - Connect the visualization to real-world phenomena and/or your idea that could help explain it, adding insight into what is happening.

How to Extract Year from a Datetime Variable in pandas

If you would like to extract the year from a datetime variable in a pandas DataFrame, you can use the `.dt.year` accessor. Below is an example:

```
import pandas as pd

# Sample DataFrame with string dates
data = {
    'Symbol': ['AAPL', 'MSFT', 'GOOG'],
    'Date': ['2024-12-29', '2024-12-30', '2025-01-03'],
    'Close': [130.21, 265.78, 122.34]
}

df = pd.DataFrame(data)

# Convert 'Date' column to datetime
```

↑ Back to top

```
df['Date'] = pd.to_datetime(df['Date'])

# Extract year from 'Date' column
df['Year'] = df['Date'].dt.year
```

This will add a new Year variable to the df DataFrame containing the corresponding year for each date.

Uploading CSV Files to Google Colab via Google Drive

When you work in **Google Colab**, your notebook runs on a temporary cloud computer.
To access files you already have in your **Google Drive** (like a CSV), you first need to **mount** your Drive.

```
from google.colab import drive
drive.mount('/content/drive')
```

What this code does

- `from google.colab import drive`
  Imports Colab's built-in tool for connecting to Google Drive.

- `drive.mount("/content/drive")`
  Connects (mounts) your Google Drive into Colab at the folder path:
  `/content/drive`

What you will see after running it

1. Colab will ask you to sign in to your Google account.
2. You will approve permission for Colab to access your Drive.
3. After that, your Drive files will appear under:

`/content/drive/MyDrive/`

Example: read a CSV file from Google Drive

```
import pandas as pd

df = pd.read_csv("/content/drive/MyDrive/path/to/your_file.csv")
```

↑ Back to top

Rubric

Project Write-up

Attribute	Very Deficient (1)	Somewhat Deficient (2)	Acceptable (3)	Very Good (4)	Outstanding (5)
1. Quality of research questions	• Not stated or very unclear • Entirely derivative • Anticipate no contribution	• Stated somewhat confusingly • Slightly interesting, but largely derivative • Anticipate minor contributions	• Stated explicitly • Somewhat interesting and creative • Anticipate limited contributions	• Stated explicitly and clearly • Clearly interesting and creative • Anticipate at least one good contribution	• Articulated very clearly • Highly interesting and creative • Anticipate several important contributions
2. Quality of data visualization	• Very poorly visualized • Unclear • Unable to interpret figures	• Somewhat visualized • Somewhat unclear • Difficulty interpreting figures	• Mostly well visualized • Mostly clear • Acceptably interpretable	• Well organized • Well thought-out visualization • Almost all figures clearly interpretable	• Very well visualized • Outstanding visualization • All figures clearly interpretable
3. Quality of exploratory data analysis	• Little or no critical thinking • Little or no understanding of data analytics concepts with Python	• Rudimentary critical thinking • Somewhat shaky understanding of data analytics concepts with Python	• Average critical thinking • Understanding of data analytics concepts with Python	• Mature critical thinking • Clear understanding of data analytics concepts with Python	• Sophisticated critical thinking • Superior understanding of data analytics concepts with Python
4. Quality of business/economic analysis	• Little or no critical thinking • Little or no understanding of business/economic concepts	• Rudimentary critical thinking • Somewhat shaky understanding of business/economic concepts	• Average critical thinking • Understanding of business/economic concepts	• Mature critical thinking • Clear understanding of business/economic concepts	• Sophisticated critical thinking • Superior understanding of business/economic concepts
5. Quality of writing	• Very poorly organized • Very difficult to read • Many typos and grammatical errors	• Somewhat disorganized • Somewhat difficult to read • Numerous typos and grammatical errors	• Mostly well organized • Mostly easy to read • Some typos and grammatical errors	• Well organized • Easy to read • Very few typos or grammatical errors	• Very well organized • Very easy to read • No typos or grammatical errors
6. Quality of Jupyter Notebook usage	• Very poorly organized • Many redundant warning/error messages • Inappropriate code to produce outputs	• Somewhat disorganized • Numerous warning/error messages • Misses important code	• Mostly well organized • Some warning/error messages • Provides appropriate code	• Well organized • Very few warning/error messages • Provides advanced code	• Very well organized • No warning/error messages • Proposes highly advanced code

↑ Back to top

Data Collection

Evaluation	Description	Criteria
1 (Very Deficient)	- Very poorly implemented - Data is unreliable.	- Poor web scraping practices with `selenium`, leading to unreliable or incorrect data from Yahoo Finance. - Inadequate use of `pandas`, resulting in poorly structured DataFrames.
2 (Somewhat Deficient)	- Somewhat effective implementation - Data has minor reliability issues.	- Basic web scraping with `selenium` that sometimes fails to capture all relevant data accurately. - Basic use of `pandas`, but with occasional issues in data structuring.
3 (Acceptable)	- Effective web scraping with `selenium`, capturing most required data from Yahoo Finance. - Adequate use of `pandas` to structure data in a mostly logical format.	
4 (Very Good)	- Well-implemented and organized - Data is reliable.	- Thorough web scraping with `selenium` that consistently captures accurate and complete data from Yahoo Finance. - Skillful use of `pandas` for clear and logical data structuring.
5 (Outstanding)	- Exceptionally implemented - Data is highly reliable.	- Expert web scraping with `selenium`, capturing detailed and accurate data from Yahoo Finance without fail. - Expert use of `pandas` to create exceptionally well-organized DataFrames that facilitate easy analysis.

↑ Back to top